



Screening — Technical Documentation

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Screening — Technical Documentation	1
1. Overview	3
2. URL, entry point, and dependencies	3
4. UI Components (in-page)	6
5. Session State	8
6. Data Layer	10
7. Business Logic	12
8. Structured Query Language (SQL) and Query Patterns	16
9. Caching	18
10. Error Handling	20
11. ACL (Access Control)	20
12. Flowcharts	21
13. Feature reference	27

Application: Market Data Portal (MDP)

Page module: `app/pages/screening.py` (~6,149 lines)

URL route: `/screening`

Registered in: `app/main.py` as `st.Page("pages/screening.py", title="Screening", url_path="screening")`

1. Overview

The Screening page implements a **Capital IQ–style progressive screener**. Users build an ordered stack of criteria (industry, geography, financial metrics, key developments, segments, estimates, forecasts). Criteria are applied sequentially to annotate a company universe; companies that fail a filter remain in the result set with **N/A** in the metric column rather than being removed (non-filtering annotation model).

Three active modes (`scr_screen_for`):

Mode	Subtitle	Results renderer
Companies	Company Screening	<code>_render_results()</code> — AG Grid
Key Devs	Key Developments Screening	<code>_render_keydevs_results()</code> — event-per-row grid
People	People Screening	<code>_render_people_results()</code> — executive compensation table

Additional modes (Equities, Fixed Income, Transactions, Projects/Portfolios) show a "coming soon" banner.

Architecture principles (from module docstring):

- All Database (DB) access delegated to `data.screening_service`
- All metric/table mappings in `data.screening_config`
- Session keys prefixed `scr_` to avoid collisions
- Criteria stack fingerprinting for cache invalidation
- Watchlists pre-filter universe before criteria run

2. URL, entry point, and dependencies

URL: `/screening` — no query-parameter deep linking; all state lives in `st.session_state` and optional saved-criteria database records.

Entry point:

```
def main():
    new_rerun_id("screening")
    _init_state()
    _flush_pending_toasts()
    _ensure_db_tables()
    _render_page_header()
    _render_screen_for()
    # ... mode-specific UI ...
    render_coresight_footer(full_width=True, stick_to_bottom=True)

main() # module bottom
```

Core / auth

Module	Symbols	Role
<code>core.auth_manager</code>	<code>require_auth</code> , <code>get_current_user</code> , <code>get_auth_data</code>	Identity via <code>main.py</code> OIDC cookie hydrate

Data — configuration

Module	Symbols	Role
<code>data.screening_config</code>	<code>OPERATORS</code> , <code>STATEMENT_CONFIG</code> , <code>PERIOD_TYPES</code> , <code>SCREENING_YEARS</code> , <code>FORWARD_SCREENING_YEARS</code> , <code>KEYDEV_CATEGORIES_ALL</code> , <code>get_metric_labels</code> , <code>get_metric_info</code> , <code>DB_SCALE</code>	Single source of truth for metrics, tables, operators

Data — service layer

Module	Symbols	Role
<code>data.screening_service</code>	<code>recompute_working_set</code> , <code>get_base_company_universe</code> , <code>apply_*_criterion</code> , <code>criteria_stack_fingerprint</code> , segment cache builders, key devs fetchers	All screening SQL and pipeline logic
<code>data.saved_criteria_service</code>	<code>save_criteria</code> , <code>get_user_criteria</code> , <code>update_criteria</code> , <code>delete_criteria</code> , grant/revoke access	Saved screening sets
<code>data.watchlist_service</code>	Create, Read, Update, Delete (CRUD) + access grants on watchlists	Watchlist persistence
<code>data.portal_users_service</code>	Portal user list	Access grant user picker
<code>data.repository</code>	<code>CompanyRepository</code> , <code>KeyStatsRepository</code> , <code>RatiosRepository</code> , <code>SegmentDataRepository</code> , <code>ExecutiveCompensationRepository</code>	Universe + tabular data + people mode
<code>data.source_router</code>	<code>get_company_source</code>	U.S. Securities and Exchange Commission (SEC) vs YFinance (service uses <code>companies_map["source"]</code> in hot path)
<code>data.segment_aliases</code>	<code>GEO_CANONICAL_GROUPS</code> , <code>canonicalize_geo_label</code> , <code>expand_geo_canonical_segments</code>	Geo segment normalization
<code>data.models</code>	<code>get_fiscal_quarter</code> , <code>parse_fiscal_year_end</code>	Fiscal Quarter (FQ) screening

UI components

Module	Symbols	Role
<code>components.styles</code>	<code>hide_sidebar</code> , <code>set_page_layout</code> , <code>render_styles</code>	Layout
<code>components.navigation</code>	<code>render_header</code> , <code>render_coresight_footer</code>	Chrome

Note: Most UI (~6k lines) is inline in `screening.py` — large `_SCREENING_CSS` , dialogs, AG Grid config, Excel builders. Only styles and navigation are shared components.

Third-party

- `streamlit` , `pandas` , `openpyxl`
- `st_aggrid` (optional; `HAS_AGGRID` flag) — filterable grid with custom `SourceLinkCellRenderer`
- `streamlit.components.v1.html` — JS Excel download

Utils

`utils.server_logger` : `new_rerun_id` , `log_structured_error` , `log_error` , `log_timing` , `log_info`

4. UI Components (in-page)

4.1 Layout regions (top → bottom)

- Page header** — Coresight branding; title from `_SCREEN_TITLES[scr_screen_for]`
- Screen For radio** — horizontal mode selector; switching resets criteria via `_reset_criteria()`
- Watchlist bar** — dropdown + Edit/Manage (`_render_watchlist_bar`)
- Dialogs** (one at a time): watchlist manager OR browse saved criteria
- Criteria palette** — Industry, Geography, Financial, Key Devs + "Saved Screenings"
- Active criterion form** — expander per type (`_render_criterion_form`)
- Status bar** — universe / working-set count (`_render_status_bar`)
- Active criteria stack** — cards with edit/remove (`_render_active_criteria`)
- Results** — mode-specific grid/table + Excel

4.2 Criteria palette (`_CRITERIA_OPTIONS`)

Label	type key
Industry Classifications	industry
Geographic Locations	geography
Financial Information	financial
Key Developments by Category	keydevs

4.3 Financial form (`_render_financial_form`)

Multi-step wizard:

1. Statement type (from `STATEMENT_CONFIG` keys + Key Stats, Ratios, Segments, Estimates, Forecasting)
2. Metric selectbox
3. Period type: Fiscal Year (FY), Calendar Quarter (CQ), FQ, TQ (trailing quarters)
4. Year / quarter selectors (single, range, trailing N)
5. Operator + threshold values (\$mm, %, or x per metric unit)
6. Optional segment member multiselect (Business / Geographical Segments)
7. Optional **Additional Data**: Credit Ratings (S&P), Store Counts (hidden child criteria)

Widget keys: `scr_fin_stmt_sel`, `scr_fin_metric_sel`, `scr_fin_period_type_sel`, `scr_fin_year_sel`, `scr_fin_quarter_sel`, `scr_fin_op_sel`, `scr_fin_val1`, `scr_fin_val2`, etc.

4.4 Dialogs

Dialog	Function	Features
Watchlist manager	<code>_dialog_watchlist_manager</code>	Create/edit name, description, companies, sectors; per-row delete; access grants
Browse saved criteria	<code>_dialog_browse_saved_criteria</code>	Load, edit, delete, view detail panel
Save criteria	<code>_dialog_save_criteria</code>	Name + description for current stack

4.5 Results grid (`_render_filterable_results_grid`)

- **AG Grid** when `HAS_AGGRID` : text filters, pinned columns (Company Name, Ticker, etc.)
- **Fallback**: `st.dataframe`
- Row selection → **Save as Watchlist** panel (`_render_save_as_watchlist_panel`)

- Segment criteria → expanded table (`_render_segment_expanded_table`) + dedicated Excel

4.6 Loading overlay

`_render_coresight_loading_overlay` — full-viewport Coresight logo + spinner during Show Results recompute (0.6s paint delay via `scr_results_pending_paint`).

4.7 Excel export

`_render_excel_download` → `_render_excel_js_download` (client-side Blob, same pattern as Market Data).

Mode	Builder	Filename
Companies (flat)	<code>_build_screening_excel</code>	<code>Screening_Results_{timestamp}.xlsx</code>
Companies (segments)	<code>_build_segment_excel</code>	same
People	inline openpyxl	<code>People_Screening_{timestamp}.xlsx</code>
Key Devs	keydevs Excel builder	event export

Module-level `_EXCEL_BUILD_CACHE` — MD5 signature keyed, bounded size.

5. Session State

All keys use `scr_` prefix unless noted.

5.1 Core (`_init_state` defaults)

Key	Type	Purpose
<code>scr_screen_for</code>	str	"Companies" default
<code>scr_active_criteria</code>	list[dict]	Ordered criterion stack
<code>scr_working_df</code>	DataFrame None	Computed universe + annotation columns
<code>scr_debug_trace</code>	list[dict]	Per-criterion timing/metadata
<code>scr_active_form</code>	str None	Open palette form type
<code>scr_show_results</code>	bool	Results section visible
<code>scr_prefill</code> / <code>scr_prefill_idx</code>	dict / int	Edit-mode prefill
<code>scr_fin_stmt</code> , <code>scr_fin_metric</code> , <code>scr_fin_operator</code> , <code>scr_fin_timeframe</code>		Financial form sub-state
<code>scr_geo_available</code>	bool None	Geography column check
<code>scr_criterion_cache</code>	dict	Per-criterion result cache
<code>scr_active_watchlist_id</code> / <code>scr_active_watchlist_name</code>		Active watchlist scope
<code>scr_watchlist_tickers</code>	set None	Ticker set (back-compat)
<code>scr_watchlist_members</code>	list[(ticker, name)]	Composite identity
<code>scr_watchlist_count</code>	int None	True company count
<code>scr_db_tables_ready</code>	bool	DDL bootstrap once
<code>scr_results_loading</code>	bool	Show Results in progress
<code>scr_results_pending_paint</code>	bool	Overlay paint delay
<code>scr_show_results_requested</code>	bool	
<code>scr_results_error</code>	str None	Last pipeline error
<code>scr_criteria_fingerprint</code>	str None	Current stack hash

Key	Type	Purpose
<code>scr_last_computed_fingerprint</code>	str None	Last successful compute hash

5.2 Watchlist dialog keys

`wl_dlg_open` , `wl_dlg_mode` , `_wl_dlg_preselect` , `wl_dlg_select` , `wl_create_*` , `wl_add_co_ms_{id}`

5.3 Saved criteria dialog keys

`scr_saved_dlg_open` , `scr_dlg_editmode_{cid}` , `scr_dlg_confirm_del_{cid}`

5.4 Fallback store

`_mem_wl_data` — in-memory watchlist when DB unreachable (offline fallback)

6. Data Layer

6.1 `screening_config.py`

Operators: Greater Than, Less Than, Between, Equals, GTE, LTE → `OPERATOR_SQL` map (Between handled separately).

Period types: `FY` , `CQ` , `FQ` , display-only `TQ` (trailing quarters: 4, 8, 12).

Scale: `DB_SCALE = 1_000_000` — UI thresholds in \$mm; SQL compares raw DB units.

STATEMENT_CONFIG — per statement type:

Field	Meaning
<code>sec_table</code> / <code>yf_table</code>	MySQL table names
<code>sec_date_col</code> , <code>sec_period_col</code> , <code>sec_period_value</code>	SEC column mapping
<code>yf_date_col</code> , <code>yf_period_col</code> , <code>yf_period_value</code>	Yahoo Finance (YF) long-format mapping
<code>metrics[]</code>	<code>{label, sec_col, yf_item, unit}</code>

Statement types in config:

- Income Statement, Balance Sheet, Cash Flow
- Key Stats (`tabular_market_data: True` → repository)

- Ratios
- Business Segments, Geographical Segments (`segment_statement: True`)
- Estimates, Forecasting (`forward-looking` year lists)

Key developments: `KEYDEV_CATEGORIES_ALL` (24 Excel-aligned DB categories), `KEYDEV_TIMEFRAMES` , alias maps for legacy migration.

6.2 `screening_service.py` — key functions

Function	Role
<code>get_base_company_universe()</code>	All companies from <code>CompanyRepository.get_companies_rows()</code>
<code>filter_universe_to_members()</code>	Watchlist pre-filter by (<code>ticker</code> , <code>company_name</code>)
<code>recompute_working_set()</code>	Full progressive pipeline
<code>apply_industry_criterion</code> / <code>apply_geography_criterion</code>	In-memory annotation
<code>apply_financial_criterion</code>	SEC/YF SQL + operator filter
<code>apply_key_stats_criterion</code> / <code>apply_ratios_criterion</code>	Repository-backed tabular
<code>apply_segment_statement_criterion</code>	Segment cache reads
<code>apply_estimates_criterion</code> / <code>apply_forecast_criterion</code>	Forward-looking tables
<code>apply_keydevs_criterion</code>	<code>coreiq_company_events</code> filter
<code>build_segment_values_cache</code>	Offline cache population
<code>get_keydevs_events_for_tickers</code>	Key Devs mode event fetch
<code>criteria_stack_fingerprint</code>	Invalidation hash

6.3 `segment_aliases.py`

- `GEO_CANONICAL_GROUPS` — United States, Canada, UK, North America, Non-US, Americas
- `canonicalize_geo_label(raw)` → canonical display label
- `expand_geo_canonical_segments(canonical)` → all raw aliases for SQL `IN` clauses
- `GEO_PENSION_SKIP_LABELS` — exclude pension rows from geo revenue
- `GEO_SUPPRESSED_DROPDOWN_LABELS` — hide duplicate raw variants in UI

6.4 `source_router.py`

```
get_company_source(ticker) -> 'SEC' | 'YFinance' | None
```

Composite tickers: `TSCO.L` → `ticker='TSCO', exchange_acronym='L'`.

Screening service splits ticker lists by `companies_map[ticker]["source"]` and runs parallel SEC/YF queries in `ThreadPoolExecutor`.

6.5 Repository methods (screening-related)

Class	Methods
<code>CompanyRepository</code>	<code>get_companies_rows()</code> , <code>get_companies_map()</code> , <code>get_all_sectors()</code>
<code>KeyStatsRepository</code>	Tabular key stats for screening
<code>RatiosRepository</code>	Tabular ratios
<code>SegmentDataRepository</code>	<code>_classify_segment_rows</code> (shared with cache build)
<code>ExecutiveCompensationRepository</code>	<code>get_sec_all_compensation()</code> , <code>get_yf_all_compensation()</code>

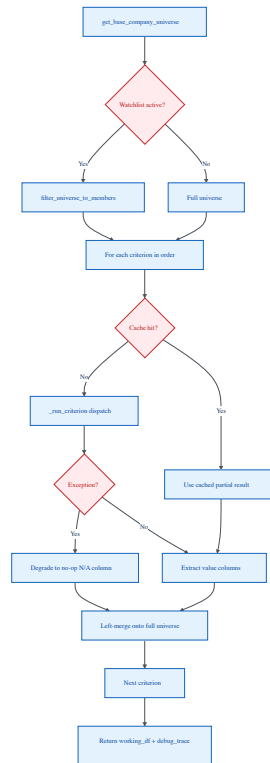
`_get_fiscal_year_end_cached(ticker)` — FQ calendar alignment (1800s TTL).

7. Business Logic

7.1 Progressive pipeline (`recompute_working_set`)

Screening progressively filters the company universe — each active criterion narrows the working set.

Screening progressively filters the company universe — each active criterion narrows the working set.



Non-filtering merge: Even if `apply_financial_criterion` returns a filtered subset internally, the pipeline extracts new columns and `merge(how="left")` onto the full universe. Failed companies show **N/A**.

7.2 Criterion dispatch (`_run_criterion` by `type`)

type	Handler
industry	<code>apply_industry_criterion</code>
geography	<code>apply_geography_criterion</code>
financial	Routed by <code>statement</code> (see below)
keydevs	<code>apply_keydevs_criterion</code>
biz_segments / geo_segments	Legacy segment filters
additional	<code>apply_additional_criterion</code> (credit ratings, store counts)

Financial `statement` routing:

Statement	Function
Income / Balance / Cash Flow	<code>apply_financial_criterion</code>
Key Stats	<code>apply_key_stats_criterion</code>
Ratios	<code>apply_ratios_criterion</code>
Business / Geographical Segments	<code>apply_*_segments_statement_criterion</code>
Estimates	<code>apply_estimates_criterion</code>
Forecasting	<code>apply_forecast_criterion</code>

7.3 Financial criterion dict shape

```
{
  "type": "financial",
  "statement": "Income Statement",
  "metric_info": {...},      # from screening_config
  "metric_label": "Total Revenue",
  "operator": "Greater Than",
  "value1": 1000.0,          # $mm in UI
  "value2": 0.0,             # Between only
  "period_type": "FY",       # FY|CQ|FQ|TQ
  "quarter": "Q1",           # CQ/FQ
  "year": 2024,              # or "Latest"
  "timeframe": "FY 2024",
  "display_col": "Total Revenue FY 2024",
  "summary": "human-readable",
  # optional:
  "num_quarters": 8,          # TQ mode
  "year_range": [2020, 2024],
  "quarter_range": {...},
  "segment_type": "business",
  "selected_segments": ["North America"],
}
```

Threshold scaling: User \$mm × `DB_SCALE` before `_apply_operator`. Equals uses \$1 raw tolerance.

7.4 Show Results flow

1. User clicks **Show Results** → `scr_show_results_requested = True`

2. `_needs_results_recompute()` compares fingerprints
3. If stale: `_trigger_recompute()` → `recompute_working_set()`
4. Loading overlay + 0.6s paint delay
5. `_render_results()` reads `scr_working_df`

Add Criteria on most types triggers immediate `_trigger_recompute()` (background); financial add may defer on failure.

7.5 Watchlist CRUD

DB tables: `coreiq_watchlists`, `coreiq_watchlist_access`, `coreiq_watchlist_companies`

Unique key: `(watchlist_id, ticker, company_name)` — disambiguates shared tickers (JD, TSCO, LULU).

Operation	Service function	Page wrapper
Create	<code>create_watchlist</code>	<code>wl_create</code>
List	<code>get_user_watchlists</code>	<code>wl_get_all</code>
Get companies	<code>get_watchlist_companies</code>	<code>wl_get_companies</code>
Add companies	<code>add_companies_to_watchlist</code>	<code>wl_add_companies</code>
Remove	<code>remove_company_from_watchlist</code>	<code>wl_remove_company</code>
Replace all	<code>replace_watchlist_companies</code>	<code>wl_replace_companies</code>
Update metadata	<code>update_watchlist</code>	<code>wl_update</code>
Delete	<code>delete_watchlist</code>	<code>wl_delete</code>
Grant/revoke	<code>grant_watchlist_access</code> , <code>revoke_watchlist_access</code>	<code>wl_grant_access</code> , <code>wl_revoke_access</code>

Access Control List (ACL) model: All watchlists visible to logged-in users; `is_owner`, `can_edit`, `can_delete` per row. Grants via `coreiq_watchlist_access`.

In-memory fallback: `_InMemoryWatchlistStore` when DB unreachable.

7.6 Saved criteria CRUD

DB tables: `coreiq_saved_criteria`, `coreiq_saved_criteria_access`

- `criteria_json` stores full `scr_active_criteria` list
- Load replaces stack + `_trigger_recompute()`

- Same owner/grant ACL model as watchlists

7.7 Fingerprints

- `_criterion_fingerprint(criterion)` — JSON excluding ephemeral fields (`summary` , `display_col` , `quarter_cols` , `year_cols`)
- `criteria_stack_fingerprint(criteria, allowed_tickers, allowed_members)` — full stack + watchlist scope

8. Structured Query Language (SQL) and Query Patterns

8.1 Company universe

```
SELECT ticker, name, name_coresight, exchange, source, exchange_acronym,
       primary_industry_coresight, country_of_incorporation
FROM coreiq_companies;
```

8.2 SEC financial latest (`_query_sec_latest`)

```
SELECT ticker, {col} AS metric_value FROM (
  SELECT ticker, {col},
         ROW_NUMBER() OVER (PARTITION BY ticker ORDER BY {date_col} DESC) AS rn
  FROM {sec_table}
  WHERE ticker IN ({sanitized_list})
         AND {period_col} = 'annual' -- or 'quarterly'
         AND {col} IS NOT NULL
) ranked WHERE rn = 1;
```

Variants: year `BETWEEN` , CQ with `QUARTER(date_col) = N` , FQ with fiscal year-end logic.

8.3 YF financial (`_query_yf_latest`)

Same `ROW_NUMBER` pattern on long-format tables:

```
WHERE line_item = '{yf_item}' AND frequency = 'annual'
```

8.4 Ticker IN safety (`_build_ticker_in_list`)

Whitelist sanitize: `[A-Za-z0-9.\-/]` , max length 20 per ticker.

8.5 Key developments

```
SELECT DISTINCT ticker FROM coreiq_company_events
WHERE ticker IN (...)
      AND event_category IN (...)
      {date_clause}
```

With headlines: `ROW_NUMBER() OVER (PARTITION BY ticker ORDER BY event_date DESC, event_id DESC)`.

8.6 Segment values cache

```
SELECT ticker, member_label, report_fiscal_year, value_mm
FROM coreiq_screening_segment_values_cache
WHERE ticker IN (...)
      AND segment_type = :type
      AND metric_key = :metric
```

Cache build: Bulk fetch `coreiq_filing_metrics_v5` per ticker chunk →
`SegmentDataRepository._classify_segment_rows` → bulk `INSERT`.

8.7 People mode compensation

```
SELECT ticker, executive_name, position, compensation_year,
       salary, bonus, stock_awards, total_compensation
FROM coreiq_executives_compensation
WHERE ticker IN (...)
ORDER BY ticker, compensation_year DESC;
```

Parallel SEC + YF fetches via `ThreadPoolExecutor`.

8.8 Database tables summary

Table	Role
<code>coreiq_companies</code>	Universe, source, sector, country
<code>coreiq_av_financials_*</code> / <code>coreiq_yf_financials_*</code>	Financial criteria
<code>coreiq_model_forecasts</code>	Forecasting criteria
<code>coreiq_av_financials_earnings_estimates</code> / YF equivalent	Estimates
<code>coreiq_filing_metrics_v5</code>	Segment source + additional data
<code>coreiq_company_events</code>	Key developments
<code>coreiq_executives_compensation</code>	People mode
<code>coreiq_screening_segment_values_cache</code>	Precomputed segment metrics
<code>coreiq_screening_segment_member_cache</code>	Segment dropdown members
<code>coreiq_watchlists</code> , <code>coreiq_watchlist_companies</code> , <code>coreiq_watchlist_access</code>	Watchlists
<code>coreiq_saved_criteria</code> , <code>coreiq_saved_criteria_access</code>	Saved screenings

9. Caching

9.1 Page-level (`screening.py`)

Function	TTL	Purpose
<code>_cached_user_criteria</code>	30s	Saved criteria list
<code>_cached_user_watchlists</code>	30s	Watchlist list
<code>_cached_portal_users</code>	300s	Access grant picker

Cleared via `_invalidate_criteria_cache()` / `_invalidate_watchlist_cache()` on writes.

9.2 Service-level (`screening_service.py`)

Data	TTL
<code>_get_segment_names_cached</code>	1800s
<code>get_all_countries</code>	3600s
<code>read_segment_member_options_cache</code>	300s
<code>get_keydevs_events_for_tickers</code>	300s
Estimates/forecast/additional helpers	1800s

9.3 Repository

Function	TTL
<code>get_companies_rows()</code>	3600s
<code>get_companies_map()</code>	3600s (derived)
<code>_get_fiscal_year_end_cached</code>	1800s

9.4 Session-level

- `scr_criterion_cache` — keyed by `(criterion_fingerprint, frozenset(universe_tickers))`
- `_EXCEL_BUILD_CACHE` — module-level MD5 keyed
- Watchlist service: 120s session TTL per user email

9.5 Startup prewarm (`main.py`)

Background thread ensures watchlist, saved_criteria, portal_users DDL before first visit.

10. Error Handling

Pattern	Behavior
<code>try/except</code> + <code>log_structured_error</code>	All major functions
User generic	<code>"Something went wrong. Please try again."</code>
Results failure	<code>"Could not load screening results. Please check logs."</code>
Segment cache miss	<code>SEGMENT_CACHE_UNAVAILABLE_MESSAGE</code> warning; background rebuild
Single criterion failure	Degrade to no-op; universe preserved
Watchlist DB down	<code>_InMemoryWatchlistStore</code> fallback
<code>DEBUG</code> enabled	Technical details in expander (diagnostics)
Toasts	<code>_queue_toast</code> / <code>_flush_pending_toasts</code> survive reruns

11. ACL (Access Control)

11.1 Page access

- Identity via `main.py` OIDC cookie hydrate; optional `require_auth(page="screening")` at page level
- No `coreiq_page_access_control` check for `"screening"`

11.2 Watchlist ACL

- All active watchlists visible to authenticated users
- Owner: full edit/delete; grants via `coreiq_watchlist_access` (`can_edit` , `can_delete`)
- UI disables actions when permissions false

11.3 Saved criteria ACL

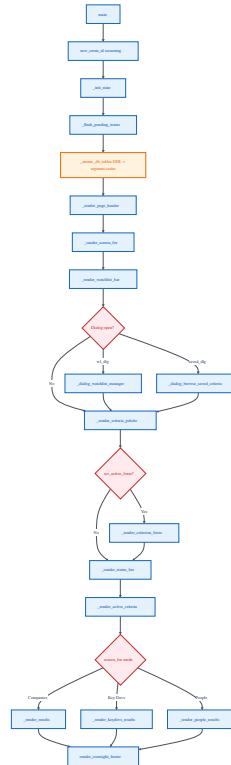
- Same owner/grant model as watchlists
- Non-owners get read-only **View** in browse dialog

12. Flowcharts

12.1 Page load

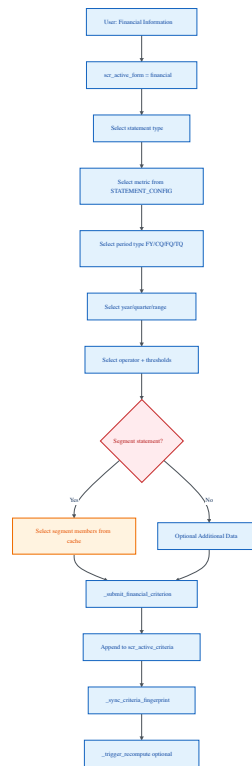
End-to-end user journeys on this page, from load through interaction to data fetch.

End-to-end user journeys on this page, from load through interaction to data fetch.



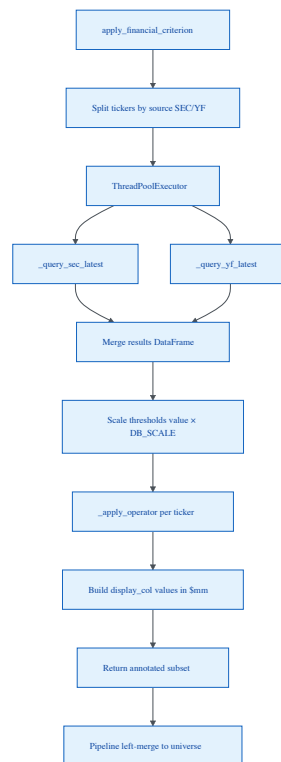
12.2 Criteria build (financial)

12.2 Criteria build (financial)



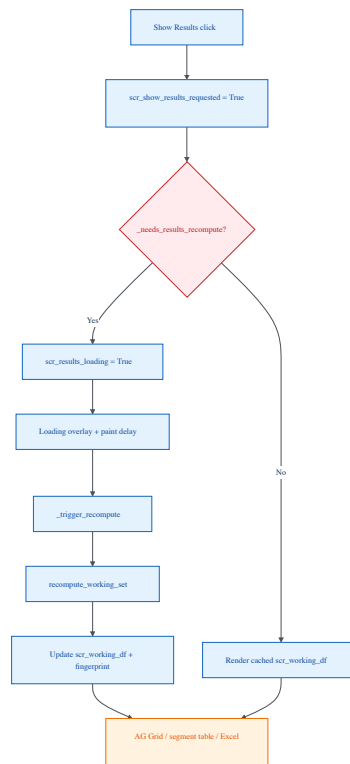
12.3 SQL execution (financial criterion)

12.3 SQL execution (financial criterion)



12.4 Show Results / recompute

12.4 Show Results / recompute



12.5 Watchlist CRUD

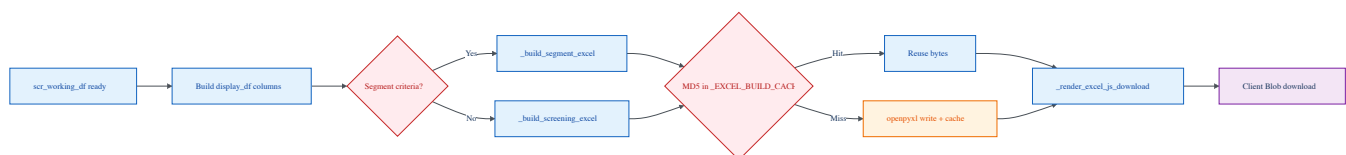
12.5 Watchlist CRUD



12.6 Export flow

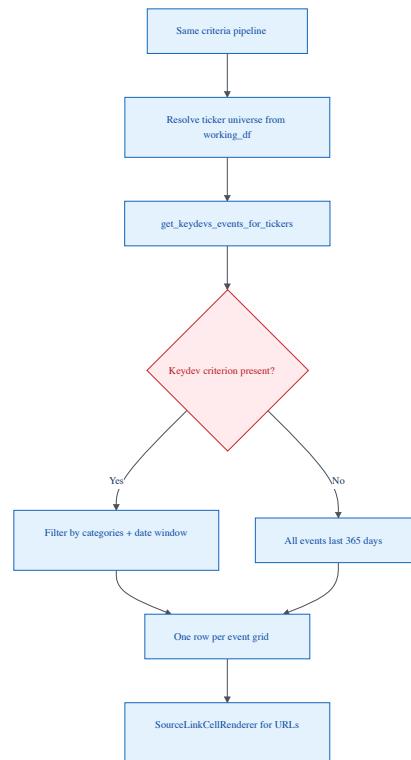
Export builds a CSV from the current working set and triggers a browser download.

Export builds a CSV from the current working set and triggers a browser download.



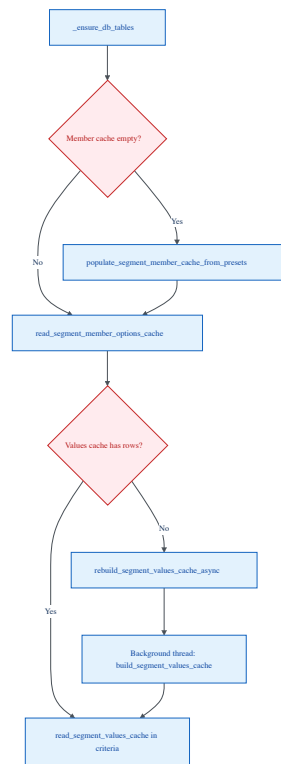
12.7 Key Devs mode

12.7 Key Devs mode



12.8 Segment cache lifecycle

12.8 Segment cache lifecycle



13. Feature reference

13.1 Companies mode

- Progressive criteria stack with non-filtering N/A columns
- Industry (in-memory sector match), Geography (in-memory country match)
- Financial: all statement types in `STATEMENT_CONFIG` + Key Stats, Ratios, Segments, Estimates, Forecasting
- Key Devs criterion annotates/filter-marks companies with matching events
- AG Grid with filters; row selection → watchlist save
- Segment expanded view + dedicated Excel when segment criteria active

13.2 Key Devs mode

- Reuses company universe from criteria pipeline
- One row per event (not per company)
- Columns: date, type, company name(s), headline, summary, sources, link

- Default: last 365 days, all categories if no keydev criterion

13.3 People mode

- Industry + geography (+ optional financial) narrow universe
- All historical executive compensation (SEC detail + YF total pay)
- Year filter dropdown; dynamic money columns
- Excel sheet "People"

13.4 Saved Screenings

- Save/load/edit/delete criteria sets with sharing grants
- Detail panel with per-filter inline edit

13.5 Segment screening specifics

- Requires `coreiq_screening_segment_values_cache`
- Geo members canonicalized via `segment_aliases`
- Metrics: Revenues, Operating Profit Before Tax, Assets, D&A, Capex
- Admin cache status in financial form; async rebuild on empty cache

Related configuration files

File	Do not duplicate logic in page
<code>app/data/screening_config.py</code>	All metric → table/column mappings
<code>app/data/screening_service.py</code>	All SQL and pipeline logic
<code>app/data/segment_aliases.py</code>	Geo canonical groups only
<code>app/data/watchlist_service.py</code>	Watchlist persistence
<code>app/data/saved_criteria_service.py</code>	Saved criteria persistence

Generated from source analysis of `app/pages/screening.py` and traced dependencies. Last reviewed against codebase structure as of project documentation pass.